

Resumenes POO

Brais Míguez Varela (pode haber fallos de ortografía)

Enero 2022

Tema 1: Encapsulación

Un programa se puede entender como una serie de instrucciones que operan sobre un conjunto de datos y generan otro conjunto de datos. Todos los lenguajes actuales tienen instrucciones de selección y de repetición de forma que es posible crear cualquier programa (*Teorama de Jacopini*). Los tipos de datos limitan las posibilidades de creación

1.1. Tipos de datos: Encapsulación

La encapsulación es una de las principales características de la POO. Asegura la integridad de los datos, limitando los trozos de código que pueden acceder a los datos del programa. La encapsulación hace uso del *Principio de ocultación*, dado que solo un trozo del código puede ver un conjunto de datos dado

1.2. Tipos de datos: Clases

En POO, los tipos de datos se conceptualizan como **Clases**. Los valores concretos de estas clases se llaman **Objetos**.

1.2.1. Clases en Java

En Java se distinguen dos especies de datos:

- **Tipos de datos primitivos**: son los mismos que en C
- **Clases**: son los tipos de datos que están vinculados a las entidades de alto nivel y, por tanto, tipos de datos que no se pueden representar directamente en el ordenador

1.2.2. Clases: Encapsulación

Las clases contienen dos tipos de código:

- **Variables o Atributos**: características que definen la entidad representada por la clase
- **Funciones o Métodos**: que acceden a los atributos para permitir lectura, escritura y proporcionar funcionalidad al programa. Las únicas funciones que pueden acceder a los atributos son las funciones de la clase.

1.2.3. Clases en Java: tipos de acceso

- **Public:**
 - **Métodos públicos:** pueden ser invocados desde cualquier clase del programa
 - **Atributos públicos:** pueden modificarse desde cualquier método del programa
- **Private:**
 - **Métodos privados:** solo pueden ser invocados desde la propia clase
 - **Atributos privados:** solo se puede acceder a ellos con métodos de la propia clase. Se usan *getters* y *setters*
- **Acceso a paquete:**
 - **Métodos de acceso a paquete:** solo pueden ser invocados desde las clases del mismo paquete
 - **Atributos de acceso a paquete:** solo pueden ser modificados por métodos de las clases del mismo paquete
- **Protected:**
 - **Métodos protegidos:** solo pueden ser invocados desde las subclases
 - **Atributos protegidos:** solo pueden ser modificados por métodos de las subclases

1.2.4. Getters y Setters

Los *getters* son métodos que **devuelven el valor** de un atributo. Solo hay un *getter* para cada atributo.

Los *setters* son métodos que **escriben el valor** de un atributo. Solo hay un *setter* para cada atributo.

1.2.5. Constructores

La declaración de un atributo de una clase tiene distintos efectos en función de su tipo de dato:

1. **Si el atributo es de un tipo de dato primitivo:**
En la declaración del atributo se realiza la reserva de memoria y se le asigna un valor por defecto
2. **Si el atributo es una clase:**
No se reserva automáticamente memoria para el atributo. Se asigna un valor inicial de *null*. La reserva se hace manualmente y para ello se invoca a los constructores a través de *new*.

Al declarar una instancia de una clase, no se reserva memoria para dicho objeto. Si no se inicializa, se genera un error de compilación.

Un constructor es un método que se invoca para reservar memoria para un objeto de la clase, reservar memoria para los atributos de dicho objeto y asignar valores iniciales. Un constructor se invoca una sola vez. Cuando se reserva memoria para un objeto, el nombre del objeto es una **referencia a la posición de memoria** en la que se guarda dicho objeto.

1.2.6. Métodos funcionales

Una vez definidos los atributos que caracterizan a la clase, se identifican los métodos de la clase que realizan las operaciones para implementar la **funcionalidad** de la clase. Un método puede tener varias implementaciones, para ello los argumentos del método deben ser distintos. En este caso, se dice que el método está **sobrecargado**

1.2.7. Almacenamiento de datos

Dependiendo del tipo de dato y de la parte del programa en la que se declara se almacena en un sitio distinto:

- **Pila(*Stack*)**: zona de memoria en la que el procesador tiene acceso a través de un puntero de pila donde la lectura/escritura es rápida y eficiente. El compilador debe saber cuanta memoria de pila se va a necesitar. Las variables existen en la pila durante la ejecución del método que las ha creado, de modo que cuando ese método termina, se libera la memoria asignada a esas variables.
Los datos que se almacenan en la pila deben tener un tamaño conocido. Se almacena todo el código correspondiente a los métodos (***Call Stack***), los datos de tipo primitivo y las referencias a los objetos creados por el programa.
- **Montón(*Heap*)**: zona de almacenamiento dinámico. En él se almacenan los objetos creados por el programa. La gestión del montón depende del *recolector de basura*.

1.2.8. Invocación de métodos

Las instrucciones de los métodos se cargan en memoria cuando se crea un objeto de la clase. A partir del momento de creación, es posible acceder a los métodos a través del operador **“.”**

1.3. Método *toString*

Es un método que devuelve la representación en texto de un objeto. Las clases heredan este método de la clase **Object**, que no aporta información sobre el contenido. Es necesario reimplementar el método para devolver una representación en texto de la forma que queremos

Tema 2: Clases y tipos de datos

2.1. Tipos de datos primitivos

Los tipos de datos primitivos son los datos predefinidos. Tienen un tamaño fijo, tienen una correspondencia directa con los tipos de datos que se pueden representar en un ordenador, la reserva de memoria se hace de forma automática y no tienen métodos.

2.2. Wrappers: Autoboxin & Unboxing

Para hacer consistente el esquema de datos de la POO, para cada tipo de datos primitivo se han definido *wrappers* que envuelven los datos primitivos en diferentes tipos de objetos

El uso de wrappers genera código que es mucho más difícil de entender. El uso del **autoboxing** y **unboxing** resuelve este problema, ya que permite tratar el wrapper como si fuese un dato primitivo, y viceversa

- **Autoboxing**: convierte un tipo de dato primitivo a un objeto de la correspondiente clase. Se usa cuando una función requiere un objeto y se le pasa un dato primitivo
- **Unboxing**: convierte un objeto de una clase wrapper al tipo de dato primitivo correspondiente. Se usa cuando un método requiere un dato primitivo y se le pasa un wrapper.

2.3. Referencias

Los nombres de los objetos son referencias a la posición de memoria de dicho objeto. El uso de referencias tiene importantes implicaciones.

Cuando se asigna un *obj_a* a otro *obj_b* en realidad no se realiza una copia de la memoria que ocupa *obj_b* en la posición de *obj_a*. En esta situación se está realizando una asignación de la referencia el *obj_a* a la referencia del *obj_b*, es decir, ambas referencias apuntan a la misma posición de memoria y la memoria que antes ocupaba *obj_a* ya no está disponible

La asignación de memoria entre dos objetos se denomina *aliasing* y es una de las principales características de los lenguajes orientados a objetos. El uso de *aliasing* evita la encapsulación de los datos, ya que permite modificar el valor de los atributos desde métodos que no pertenecen a las clases que contienen dichos atributos.

Contrapartes del aliasing

- Los programas son mucho más difíciles de mantener, porque los valores de los atributos de tipo objetos se podrían modificar en cualquier parte del código sin ningún tipo de control por parte de la clase a la que pertenecen

Evitar el *aliasing* en lenguajes de POO reduciría enormemente su rendimiento, puesto que la asignación entre dos objetos sería una copia completa de una zona de memoria a otra. Es muy difícil el desarrollo de programas en los que no se use el *aliasing* en alguna parte del código. Es necesario evitar aquellas partes del código en las que se debe evitar el *aliasing*. La única forma de evitarlo es introducir manualmente código que genere una nueva referencia al objeto, es decir, reserve memoria y copie los atributos del objeto, generando nuevas referencias cuando sea necesario. Para facilitar esto, Java tiene el método *clone*

2.3.1. Método *clone*

Este método está pensado para **generar una copia exacta** de un objeto, almacenando esa copia en una posición de memoria distinta. No es práctico implementar *clone* en todas las clases, sobretodo cuando se quieren **copias profundas**. En una copia profunda es necesario reservar memoria para todos los atributos de la clase, incluyendo **todos los elementos de un conjunto de datos**.

2.3.2. Tipos de referencias en Java y usos principales

1. Referencias fuertes:

- Son las referencias por defecto
- Se crean cuando se instancia un objeto
- El recolector de basura no las elimina hasta que apuntan a *null*

2. Referencias débiles:

- Para crear estas referencias, se debe instanciar a la clase *WeakReference*, teniendo como argumento la referencia fuerte a la que apunta.
- La creación de una referencia débil no obliga al recolector de basura a mantener la referencia fuerte a la que apunta
- Para acceder a la referencia fuerte, se puede usar el método *get()* pero no asegura que devuelva una referencia fuerte no nula
- El recolector elimina las referencias débiles cuando no apuntan a una referencia fuerte o la referencia fuerte apunta a *null*

3. Referencias suaves:

- Para crear este tipo de referencias, se debe instanciar la clase *SoftReference*, teniendo como argumento la referencia fuerte.
- Aunque se elimine la referencia fuerte, se puede seguir accediendo a esa posición de memoria
- Son eliminadas cuando se necesita memoria

4. Referencias fantasma:

- Para crear estas referencias, se debe instanciar la clase *PhantomReference*, con dos argumentos: el primero es la referencia fuerte y el segundo es una instancia de la clase *ReferenceQueue*
- A pesar de que se elimine la referencia fuerte, se se puede seguir accediendo a ella, ya que se almacena en la cola que se pasó como argumento.
- El método *get()* devuelve siempre *null*, ya que están pensadas para acceder a memoria cuando las referencias fuertes ya no están disponibles.

Las referencias fantasmas y sueaves se usan para **hacer cachés de memoria**, de manera que se puedan acceder a referencias fuertes que ya no estén disponibles. Las referencias débiles se usan para acceder dinámicamente a la referencia fuerte de un objeto, evitando referencias indiscriminadas de dicho objeto (*aliasing*)

2.4. Clase String

Las cadenas de texto se pueden crear de dos formas distintas:

1. **Directamente**: asignando una cadena de texto al objeto, en cuyo caso se almacena en una zona del montón llamada *String Pool*, de modo que **cada vez que se asigna la misma cadena de texto, se apunta a la dirección de memoria que la contiene**
2. **Indirectamente**: el objeto se crea usando un constructor de String, en cuyo caso se almacena en el montón pero fuera del *String Pool*. Cuando se crea un String de esta forma se reserva memoria aunque tenga el mismo valor que una cadena previa

String es una clase de Java, pero no se comporta como el resto. Funciona igual cuando se realizan asignaciones de dos strings. Una cadena de texto es un objeto inmutable, es decir, una vez que se asigna memoria, no se puede modificar. El uso de *aliasing* con strings puede llevar a una **merma importante del rendimiento**

2.5. Método *equals*

Dos objetos se consideran iguales si ocupan la misma posición de memoria o si tienen el mismo tipo y sus atributos tienen los mismos valores. *Equals* es un método informa de si dos objetos son iguales y está en todas las clases. El método que se hereda, compara direcciones de memoria, por lo que es necesario reimplementarlo para comprobar los atributos que nos interesan

Tema 3: Conjuntos de datos

Una buena parte de la programación consiste en manejar conjuntos de datos sobre los que se realizan operaciones CRUD de búsqueda, inserción, modificación y borrado. La mayoría de los lenguajes de programación soportan la representación de conjuntos de datos mediante el concepto de **array**

3.1. Arrays

Un array es un conjunto de elementos del mismo tipo que ocupan posiciones de memoria consecutivas para facilitar su acceso de forma sencilla a través del índice. En Java, un array de datos es un objeto, por lo que se debe usar *new* y el número máximo de elementos para reservar memoria y tiene como atributo público la longitud del array. Son las únicas estructuras que permiten almacenar tipos de datos primitivos. Para acceder a las posiciones de un array usamos un índice. Si intentamos acceder a una posición que no existe, se genera un error **ArrayIndexOutOfBoundsException**.

Los arrays tienen importantes **limitaciones** ya que tienen un tamaño fijo y no se pueden borrar datos cuando son tipos primitivos. Si se quiere borrar un tipo objeto, se pone el valor *null* en la posición en la que se quiere eliminar.

3.2. Colecciones

3.2.1. Collection

Las colecciones son **grupos o conjuntos de datos o elementos** sobre los que se definen operaciones de inserción, borrado y actualización. *Collection* es una interfaz que define las operaciones, por tanto no se puede reservar memoria para un objeto de este tipo. **No se puede recorrer** una colección a través de un índice, ya que no es un grupo ordenado de elementos

Los métodos *contains()* y *remove()* hacen uso del método *equals* para comprobar si el objeto se encuentra en la lista. Algunas implementaciones de la interfaz no soportan todos los métodos, generando una excepción del tipo **UnsupportedOperationException**

Una de las formas de recorrer una colección es a través de un *for-each*. Las colecciones no se pueden modificar mientras se están recorriendo, porque se genera un error del tipo **ConcurrentModificationException**

3.2.2. Iterator

Es una interfaz en la que se definen un conjunto de operaciones que se pueden usar para **recorrer los elementos** de una colección. Para recorrer todos los elementos, se usa un **puntero o iterador** que señala al siguiente elemento de la colección, de modo que las operaciones básicas son:

- **hasNext()**: indica si existe un elemento en la siguiente posición
- **next()**: obtiene el elemento que se encuentra en la posición siguiente y **actualiza** el puntero a la posición siguiente
- **remove()**: elimina el elemento que está en la posición actual

Si se usa un iterador en un bucle, se debe invocar una sola vez, ya que si se invoca varias veces, se actualiza el puntero tantas veces como se ha invocado.

Diferencias entre un *for-each* y un *iterator*:

- Un iterator permite la eliminación de elementos mientras que se recorre, eliminando **tanto los elementos de la colección como del iterator**
- Un iterator solo se puede usar una vez para recorrer los elementos, ya que el puntero iterator habrá llegado al final de la colección y *hasNext()* devolverá siempre **false**
- El **rendimiento** de ambos es igual

3.3. Listas

3.3.1. Lists

Las listas son colecciones en las que los **elementos tienen un orden** que está relacionado con la secuencia en la que dichos elementos han sido introducidos. Una lista mantiene siempre el orden y asigna un índice a cada posición. *List* es un interfaz que define todas las operaciones de una lista. Las listas se pueden recorrer usando los índices que se generan en el momento de inserción. Una lista puede contener objetos duplicados

3.3.2. ArrayLists

La clase *ArrayList* es un tipo de lista que soporta el **redimensión dinámica** cuando el número de datos que se desea almacenar es mayor que la capacidad de almacenamiento de la lista. Si el redimensionamiento se realiza con mucha frecuencia, **penalizará el rendimiento** de la lista. Una instancia de *ArrayList* puede contener objetos del tipo *null*

Si la lista se recorre a través de un índice, es posible modificar el contenido. Al recorrer la lista, es necesario comprobar que los objetos almacenados son distintos de *null*. La clase *ArrayList* encapsula un array de objetos al que se accede mediante los métodos de *List*. Las operaciones de acceso a los objetos del *ArrayList* tienen lugar en **tiempo lineal** con el número de objetos almacenados en el array $\Rightarrow O(n)$

El *aliasing* tiene un mayor impacto al usar conjuntos de datos, ya que se pueden modificar sus elementos sin cambiar la dirección del conjunto

3.4. Conjuntos

3.4.1. Set

Los conjuntos son colecciones en las que **no se repiten los datos**, es decir, todos los datos son diferentes de acuerdo con el método *equals*. *Set* es un interfaz en el que se definen las mismas operaciones que en una **Collection**. Las implementaciones de *Set* deberán asegurar que para cada par de objetos de la colección se cumple la condición de que son distintos. En un *Set* puede haber un elemento *null* como máximo. Se puede usar un *iterator* o un *for-each* para recorrerlo.

Como el criterio de igualdad es crítico, se obliga a **sobrescribir el método *equals***. También se debe controlar que la modificación de un objeto no dé como resultado que dicho objeto sea igual a alguno de los elementos del conjunto

3.4.2. *HashSet*

Un `HashSet` es un wrapper de la clase `HashMap`, de modo que internamente los datos se almacenan como las claves de un objeto `HashMap`. No se asegura que los datos se guarden en el mismo orden en el que se insertan. Las operaciones de acceso a los objetos del `HashSet` tienen lugar en tiempo constante $\Rightarrow O(1)$

3.5. Mapas

3.5.1. *Maps*

Un `Map` es un interfaz que relaciona una clave con un valor, de manera que a partir de la clave se obtiene el valor que está asociado a ella. Tanto las claves como los valores tienen que ser objetos. Las claves no pueden estar repetidas. Los objetos si pueden estar repetidos. Algunas implementaciones permiten almacenar claves y valores nulos.

La clave de un mapa puede ser cualquier tipo de objeto. Para localizar y obtener la clave con la que se recupera el valor se hará uso de `equals`, que deberá estar sobrescrito para la clase a la que pertenece la clave. La clave debería ser un objeto inmutable

Algunas implementaciones de la interfaz `Map` no soportan todas las operaciones, por lo que se genera un error del tipo `UnsupportedOperationException`

Los datos almacenados en un mapa no se pueden recorrer directamente, ya que el objetivo de los mapas es acceder a los datos a partir de las claves

`Map.Entry` es un interface que define métodos para acceder a una entrada de un mapa, es decir, a la dupla $\langle \text{clave}, \text{valor} \rangle$

3.5.2. *HashMap*

Los objetos que son las claves del mapa están asociados a un código hash que se usa para comprobar si dos claves son iguales, complementando el método `equals`. Cuando se comparan dos objetos de un `HashMap`, primero se comprueba si los objetos tienen el mismo hash code, de modo que si no es el mismo se consideran distintos; si el hash code es igual, se invoca al método `equals` para aplicar el criterio de igualdad y comprobar que los objetos son iguales.

El uso de hash code simplifica y hace más eficiente el acceso a los datos en objetos `HashMap` ya que primero se compara entre enteros y solo en el caso de que sean iguales tiene lugar una comparación más compleja.

La clase `HashMap` implementa una tabla hash en la que los datos se almacenan de forma desordenada de acuerdo con una función hash. Las operaciones de acceso a los elementos de un `HashMap` tienen lugar en tiempos constantes $\Rightarrow O(n)$

3.6. Conjuntos de datos: Comparativa

- Las listas ocupan menos memoria que los mapas
- Los mapas son más rápidos que las listas

Tema 4: Herencia

Se puede definir la herencia como el mecanismo en virtud del cual una **clase derivada** reutiliza los atributos y los métodos que pertenecen a una **clase base**. El objetivo de la herencia es la construcción de clases basadas en otras clases que **ya tienen implementada** la conducta deseada. Los constructores no serán heredados por las clases derivadas, ya que se considera que son **específicos** de la clase a la que pertenecen. Una clase derivada se considera una extensión de la clase base en la que los atributos y los métodos pueden ser heredados o pueden estar definidos **en la propia subclase**.

Tipos de herencia

- **Herencia simple:** Una clase B hereda los atributos y métodos de una clase A. Es el tipo de herencia más básico.
- **Herencia multi-nivel:** Una clase C hereda los atributos y métodos de una clase B que, a su vez, hereda de la clase A, de modo que la clase C hereda atributos y métodos de la clase A.
- **Herencia Jerárquica:** Las clases B y C heredan los atributos y métodos de la clase A, de modo que ambas se diferencian por sus métodos propios.
- **Herencia Múltiple:** La clase C hereda los métodos de la clase A y de la B, de modo que es necesario definir políticas de herencia cuando A y B tienen métodos comunes.

El principal beneficio de la herencia es la **reutilización de código**, en la que un trozo que ha sido previamente desarrollado, depurado y validado, se usa en otra clase sin ser necesaria una modificación adicional. También facilita el mantenimiento y extensibilidad de los programas y simplifica el código. También genera desventajas como por ejemplo, si el nivel de jerarquización es muy profundo el código se vuelve complicado de entender, los cambios en las clases base tienen grandes impactos en las clases derivadas y, para facilitar la herencia, es posible que los atributos dejen de ser privados.

4.1. Composición

La composición es el mecanismo mediante el cual una clase contiene objetos de otras clases a los que se le delega ciertas operaciones para conseguir una funcionalidad dada. El principal beneficio es que **reparte las responsabilidades entre los objetos**, es decir, cada objeto se encarga de realizar unas operaciones sobre los atributos, que después podrán invocar a los demás objetos. También facilita el mantenimiento de los programas y facilita la extensibilidad. Otro beneficio importante es que las clases están más desacopladas entre sí, de modo que un cambio en una de ellas tiene un impacto reducido en las otras.

Caso	Diseño basado en herencia	Diseño basado en composición
Inicio del desarrollo	Más rápido	Más lento
Diseño del software	Más sencillo	Más complejo
Efectos no deseados	Ocurren con más frecuencia, sobretodo cuando se trata de jerarquías profundas	Se reducen y están más localizados, en los métodos delegados
Adaptación a cambios	Para jerarquías profundas y con múltiples sobrescrituras es más complicado	Más sencillo de cambiar, puesto que solo afecta a las clases delegadas
Validación	Difícil de realizar	Más sencillo
Extensibilidad	Complicado en jerarquías profundas	Más sencillo a través de la composición de clases

4.2. Herencia en Java

En la herencia de Java, se hace énfasis en que una clase derivada extiende los atributos y métodos de una clase base, aunque se ponen restricciones entre los tipos de clase. Solo se heredan los atributos y métodos que son públicos para la clase derivada, es decir, se hereda en función de los tipos de acceso:

- **Private:** la clase derivada nunca heredará los atributos y métodos
- **Public:** la clase derivada siempre heredará los atributos y métodos
- **Acceso a paquete:** la clase derivada solo heredará los atributos y métodos de la clase base si se encuentra en el mismo paquete
- **Protected:** la clase derivada siempre heredará los atributos y métodos

Se podría considerar que sería mejor que los atributos fuesen públicos o protegidos para facilitar la herencia, pero **deben ser privados** porque se considera que la encapsulación es una característica que ofrece mayores beneficios que la herencia

En realidad, la clase derivada hereda **todos los atributos y métodos**, independientemente de su tipo de acceso, pero si son privados **se heredan pero no se puede acceder a ellos**.

Los constructores de la clase base no se heredan ya que se considera que contienen código específico de la clase a la que pertenecen. Si se implementa un constructor sin argumentos para la clase derivada, **obligatoriamente** deberá implementarse un constructor sin argumentos para la clase base, en caso contrario, se genera un error. Si se implementa un constructor con argumentos en la clase derivada, se puede acceder al constructor de la clase base mediante *super()*. *Super* solo puede acceder a elementos de la clase base que es **inmediatamente superior** a la clase derivada

4.3. Métodos y sobrescritura

La sobrescritura de métodos es un mecanismo mediante el cual un método heredado de una clase base se **implementa nuevamente** en la clase derivada. El nombre, argumentos y tipo que devuelve deben ser los mismos en todas las clases mientras que el tipo de acceso puede ser el mismo o superior. La sobrescritura es necesaria cuando la implementación de un método de la clase base **no es válida** en la clase derivada o es conveniente adaptarla a las **características**

específicas de la clase. Todas las clases que se crean en un programa de Java son derivadas de la clase *Object*

4.4. Clases abstractas

Las clases abstractas son clases que no se pueden instanciar. Pueden tener constructores pero no se pueden invocar a través de *new*, de modo que únicamente se pueden invocar mediante *super* cuando son las clases bases de otras clases. Pueden tener atributos y métodos. Se suelen usar como clases base para otras clases que si que se pueden instanciar.

Las clases abstractas deben tener implementados la mayor cantidad posible de métodos para que las clases derivadas puedan heredarlos y favorecer la reutilización del código. Una clase debería ser abstracta cuando **todos los objetos** pertenecen a cualquiera de las otras clases de la jerarquía, no siendo necesario crear un objeto de la clase abstracta

4.4.1. Métodos abstractos

Son métodos que no tienen cuerpo, es decir, métodos que no están implementados y en los que solamente se especifica su nombre, lo que devuelve y los argumentos. Si una clase tiene un método abstracto, entonces debe ser necesariamente una clase abstracta. Los métodos abstractos deben ser implementados en las clases derivadas de la clase abstracta a la que pertenecen, **siempre que dichas clases no sean abstractas**

4.5. Clases, atributos y métodos finales

Una jerarquía de clases se construye con clases **abstractas** (primeros niveles), con clases **instanciables** (niveles medios y finales) y con clases **finales**. Las clases finales son aquellas de las que no se pueden crear clases derivadas, es decir, son nodos de la jerarquía de clases. Una clase debería ser final cuando se garantiza que no se deben crear clases derivadas de ella

La palabra **final** en Java indica que un elemento no se puede modificar. Si es un método, no se puede sobrescribir en una clase derivada. Si es un atributo, cuando se le da un valor, no se puede cambiar. Los atributos finales se usan para implementar las **constantes** del programa. Típicamente una constante se define con **final static**, ya que así es posible hacer uso de la constante sin que sea necesario crear un objeto de la clase. La palabra **static** usada como modificador en atributos o métodos hace que se almacenen en la memoria estática, permitiendo que estén disponibles desde el inicio del programa

Tema 5: Polimorfismo

El polimorfismo en herencia es el mecanismo mediante el cual un objeto se puede comportar de múltiples formas en cada momento, en función del contexto en el que aparece dicho objeto dentro del programa. Un objeto se puede comportar como **una instancia de la clase a la que pertenece o una instancia de las clases de nivel superior**. El polimorfismo está relacionado con la herencia y con el uso de los métodos de las clases que se heredan/sobreescriben.

Las clases abstractas no solamente se usan para estructurar las jerarquías de clases, sino que también para restringir los métodos que pueden invocar los objetos.

5.1. Upcasting & Downcasting

Cuando se indica que un objeto se comporta como una clase de la jerarquía diferente a la que pertenece, se está forzando un cambio en el tipo de dato del objeto:

- En **upcasting** un objeto de una clase derivada se comporta como una de las clases bases de la jerarquía
- En **downcasting** un objeto de una clase base se comporta como una de las clases derivadas de la jerarquía

Este cambio del tipo de dato no implica una nueva reserva de memoria para el objeto, sino que el objeto seguirá siendo el mismo, independientemente de los cambios de tipo de dato que tengan a lo largo del programa. El casting de objetos no solamente conlleva un cambio en el tipo de dato, si no que también afecta a la accesibilidad de los atributos y de métodos que se pueden invocar desde el objeto.

5.1.1. Upcasting

Es el mecanismo más habitual con el que se facilita el polimorfismo, ya que parece natural pensar que un objeto de la clase derivada se comporta como una de sus clases bases. Cuando se realiza un *upcasting* no es necesario indicar de forma explícita un cast. Los métodos y atributos que son propios de la clase a la que pertenece el objeto no son accesibles, es decir, se pierde la visibilidad de esos métodos y atributos, puesto que el objeto deja de comportarse como esa clase.

Si un método está sobrescrito, cuando se realiza un upcasting el objeto usará la implementación del método correspondiente a **la clase a la que pertenece el objeto**, de modo que aunque se comporta como otra clase, no utiliza las implementaciones de la otra clase.

Upcasting es un **concepto clave** en programación orientada a objetos, puesto que facilita la toma de decisiones en relación al diseño del programa. Si en un programa es necesario el upcasting, entonces **se deberá aplicar el mecanismo de herencia**.

El mecanismo de upcasting nunca producirá un error, ya que con la herencia se garantiza que las clases derivadas tienen los mismos métodos que la clase base. Una vez se realiza upcasting sobre un objeto, no hay forma de acceder a los métodos específicos de la clase a la cual pertenece dicho objeto. Para aprovechar todo el potencial del upcasting el diseño debe ser **muy cuidadoso**. Las clases abstractas permiten definir qué métodos son necesarios en las clases que derivan de ellas, facilitando con ello el uso del upcasting para la simplificación de los programas.

5.1.2. Downcasting

Con downcasting se está forzando el comportamiento de los objetos en las jerarquías de las clases, puesto que por lo general no se puede asegurar que un objeto de una clase base se comporta como uno de una clase derivada. Usar downcasting no es una operación segura. Típicamente se usa cuando se necesita deshacer el resultado del upcasting, es decir, cuando se quiere recuperar la asignación del objeto a la clase a la cual pertenece.

Cuando se realiza downcasting es necesario indicar **de forma explícita** a qué clase derivada “convertirá” el objeto de la clase base. Se usa un **cast** con el que se fuerza la conversión de los tipos de datos desde la clase base a la clase derivada, teniendo el siguiente formato:

(<Nombre de la clase derivada>)

El compilador no comprueba si el cast **tendrá lugar de forma correcta** ni si el objeto se encuentra en la misma jerarquía que la clase derivada a la que se “convierte”

El principal problema del downcasting es que **no se puede garantizar** que el objeto disponga de los mismos métodos y atributos que los de la clase derivada a la cual se realiza el casting. La operación de casting tiene lugar durante **tiempo de ejecución**

El operador **instanceof** determina en tiempo de ejecución si el tipo de dato de un objeto es una clase dada, de manera que si devuelve cierto, se podrán invocar los métodos de dicha clase

5.2. Beneficios del polimorfismo

- Facilita la **simplificación del código del programa**, puesto que los objetos se pueden manejar como objetos de las clases base, de modo que la invocación de los métodos comunes a todas las clases se realiza desde una misma clase.
- Facilita la **extensibilidad de los programas**, ya que la inclusión de nuevas clases en la jerarquía que implementa métodos comunes a las clases base, no supondrá modificaciones en el código del programa. Con **downcasting** la introducción de nuevas clases podría implicar cambios en el código del programa, pero serían extensiones

Tema 6: Interfaces

Las interfaces son uno de los componentes clave en el **diseño de los programas**, ya que con ellos se establecen de forma muy precisa los requisitos que deben cumplir las clases del programa. Se **definen los tipos de datos** que se deben crear en el programa, de modo que, a pesar de existir otros, los de las interfaces tienen que estar definidos. Se **definen exactamente los métodos** que deben tener las clases del programa, indicando exactamente sus nombres, argumentos y tipos de datos que devuelven. Los métodos de interés son los que estén en las interfaces

Una interfaz se entiende como un **compromiso o acuerdo** entre programadores de aplicaciones para que las clases que creen unos se puedan usar sin errores ni mayores adaptaciones por otros. El objetivo de un interfaz es establecer los métodos que una clase debe implementar y que serán los **métodos visibles y accesibles** por las otras clases del programa.

El uso de interfaces facilita **el desarrollo y el mantenimiento de los programas**, puesto que con ellas se dividen las responsabilidades y permiten una implementación independiente de las clases. Un cambio en una clase en la que se implementan los métodos de la interfaz, **no afectará a las otras clases** que hacen uso de dicha interfaz. Pero un cambio en la interfaz, **afectará de forma significativa** a las clases que implementan la interfaz y a las clases que la usan

Una interfaz se entiende como **una plantilla de una clase** que contiene constantes, declaraciones de métodos abstractos, métodos por defecto y métodos estáticos. No se pueden instanciar por lo que no tienen constructores. Las clases deben implementar los métodos abstractos de la interfaz para que esta pueda ser usada en otras clases. Una vez la interfaz haya sido implementada, **se comportará como una clase**, pudiendo invocar a los métodos.

6.1. Interfaces en Java

- Los atributos de una interfaz son implícitamente públicos, estáticos y finales
- Los métodos de una interfaz son implícitamente públicos y abstractos
- Los métodos por defecto solo se pueden definir en las interfaces y son heredados directamente por las clases que las implementan
- Los métodos estáticos también se implementan en el interfaz y son accesibles directamente desde cualquier clase

Si una clase implementa una interfaz, entonces esa clase tendrá los mismos métodos que el interfaz, del mismo modo que una clase derivada tiene los mismos métodos que una clase base. La clase que implementa una interfaz se puede entender como una clase derivada de la interfaz. Se pueden aplicar todos los conceptos de herencia, jerarquías de clases, clases abstractas y polimorfismo

Clases abstractas vs Interfaces

Clases abstractas	Interfaces
Se definen cuando las clases derivadas tienen algunos métodos comunes	Se definen cuando las clases que los implementan tienen diferentes implementaciones para diferentes objetos
La herencia múltiple no es posible	Se puede entender que la herencia múltiple es posible, aunque solo puede haber una clase base
Pueden tener métodos abstractos y métodos concretos	Solamente tienen métodos abstractos, además de métodos estáticos y métodos por defecto
Los métodos abstractos pueden ser públicos y protegidos	Los métodos abstractos deben de ser públicos
Tienen constructores	No tienen constructores
Pueden tener cualquier tipo de atributo	Los únicos atributos que puede tener son de tipo estático y final (son constantes)

6.2. Métodos por defecto

El problema de las interfaces es la incapacidad de las clases que las implementan para adaptarse a los cambios que tendgan lugar en ellos. Al añadir o actualizar métodos que están en una interfaz, las clases que las implementan **generaran errores de compilación**. Los **métodos por defecto** resuelven parcialmente el problema de la adaptación de las clases a los cambios en las interfaces que implementan

Los métodos por defecto son métodos que se **deben declarar e implementar** en la interfaz, de modo que se heredarán por las clases que implementan dicha interfaz. La implementación debe operar únicamente con los argumentos del método por defecto, ya que en el interfaz no existen atributos que no sean constantaes. Los métodos por defecto **pueden ser sobreescritos** en las clases que implementan el interfaz o en las clases derivadas de ellas. Estos métodos son implícitamente públicos y, desde ellos, se pueden invocar a los métodos abstractos de interfaz, que son implementados por las clases.

Se debe valorar la conveniencia de definir métodos por defecto, sobre todo si necesitan argumentos asociados a atributos de las clases que implementan las interfaces que contienen dichos métodos. El código se hace más complejo de entender y de mantener, puesto que los métodos de las clases tienen argumentos que no serían necesarios al hacer referencia a los atributos de dichas clases.

6.3. Métodos estáticos

Los métodos estáticos son métodos que pueden ser invocados desde el inicio del programa, de modo que para invocarlos **no es necesario crear ningún objeto de la clase** en la que se han definido. Se pueden definir en cualquier clase o en cualquier interfaz. Se cargan automáticamente en memoria. En un método estático solo se pueden invocar métodos de la clase o del interfaz que sean estáticos, puesto que tienen que estar disponibles desde el inicio del programa. No son heredados por las clases e interfaces

Métodos estáticos vs por defecto

Métodos por defecto	Métodos estáticos
Solamente pueden estar definidos en los interfaces	Se pueden definir en interfaces y en clases
Pueden invocar cualquier tipo de método, incluyendo métodos por defecto, estáticos y abstractos	Solamente pueden invocar métodos que, a su vez, sean métodos estáticos
Son heredados por las clases y por los interfaces derivados del interfaz que los contiene	No pueden ser heredados, siendo métodos que son específicos de las clases o de los interfaces en los que están implementados
No están disponibles en el arranque del programa, es necesario crear un objeto para invocarlos	Están disponibles al arrancar el programa

6.4. Herencia e interfaces

Si una clase abstracta cumple las mismas condiciones que una interfaz, es decir, solamente tiene métodos abstractos y no tiene ningún atributo que no sea constante, entonces se podría pensar que una clase derivada tiene varias clases base, o lo que es lo mismo, existe herencia múltiple.

Una clase debe implementar todos los métodos abstractos de todas las interfaces que implementa y hereda todos los métodos por defecto de todas esas interfaces.

- Si varias interfaces tienen en común algún método abstracto, la implementación de ese método en la clase será válida para todas las interfaces
- Si varias interfaces tienen en común algún método estático, no existirá conflicto en la clase, ya que ese tipo de métodos no se heredan
- Si varias interfaces tienen en común algún método por defecto, existirá una colisión entre dichos métodos en relación a cuál de ellos será el que heredará la clase
Para resolver este conflicto, la clase que implementa las interfaces deberá de sobrescribir los métodos por defecto que son comunes a ambas interfaces. En la propia sobrescritura de los métodos por defecto se puede realizar la selección de la implementación de los métodos por defecto, utilizando para ello `super`

Se pueden crear jerarquías de interfaces siguiendo las mismas reglas de herencia que en caso de jerarquía de clase. Una interfaz derivada puede tener varias interfaces base. Las interfaces derivadas heredarán todos los métodos de las interfaces base, excepto los estáticos. Existe sobrescritura de los métodos por defecto. No se puede establecer una herencia entre clases e interfaces, es decir, una clase no puede extender una interfaz.

Las jerarquías combinan la herencia de clases, la herencia de interfaces y la implementación de interfaces por parte de las clases. Las jerarquías resultantes pueden ser muy complejas, pero lo importante es que sean extensibles de forma relativamente sencilla

Tema 7: Excepciones

Una de las tareas que es necesaria realizar en el desarrollo de un programa es la **gestión de los errores** que ocurren durante su ejecución. Una **excepción** es la ocurrencia de errores y/o situaciones de interés durante la ejecución de los métodos que proporcionan funcionalidad al programa. Las excepciones también son situaciones a las que se le debe dar respuesta y que forman parte de la lógica del programa.

El mecanismo de Java para el tratamiento de las excepciones aplica la filosofía de **separar** el código para la detección, la comunicación y el tratamiento de las excepciones del código de los métodos que proporcionan funcionalidad al programa. Una excepción se especifica como una clase derivada de la clase *Throwable*.

Tipos de excepciones en Java

- **Excepciones chequeadas:** se comprueba en tiempo de compilación en qué métodos puede ocurrir una excepción, en cuyo caso **esos métodos deben indicar de forma explícita**. Son clases derivadas de la clase *Exception*. Si el error que provoca la excepción no impide la ejecución del programa, la excepción debería ser de este tipo.
- **Excepciones no chequeadas:** no se comprueba en tiempo de compilación cuáles son los métodos en los que tienen lugar estas excepciones, por lo tanto es responsabilidad del programador chequear donde puede ocurrir para capturarlas y tratarlas. Son clases derivadas de las clases *RuntimeException* o *Error*.

La definición de excepciones como clases derivadas de *Exception* otorga a los programadores un mayor control sobre la gestión de dichas excepciones, puesto que obliga a los métodos a declarar que excepciones podrán ocurrir durante su invocación. Para las excepciones chequeadas, el compilador exige que se indiquen **explícitamente** los métodos en los que tienen lugar dichas excepciones, es decir, se deben etiquetar todos los métodos que pueden lanzar una o varias excepciones.

En la signatura del método se deben indicar **todas las excepciones** que podrían ocurrir durante la ejecución del método, es decir, todas las excepciones que se generarán. Si las excepciones pertenecen a la misma clase base, sería suficiente con etiquetar el método con la clase base.

7.1. Gestión de excepciones

7.1.1. Intentar

La gestión de las excepciones comienza con la invocación de un método_A desde otro método_B de forma que **el método_B intenta ejecutar con éxito el método_B**. Es necesario especificar en qué parte del código se va intentar la ejecución con un *try{}catch{}finally{}* . En el código puede haber tanto bloque *try{}catch{}finally{}* como invocaciones a métodos que generan excepciones.

7.1.2. Lanzar

Una vez se invoca la ejecución del método_B el control del programa pasa al código de dicho método que tiene dos partes: *la detección y la generación* de la excepción para ser tratada.

La detección consiste en especificar dentro del código del método las condiciones en las cuales se genera excepción. Al generar la excepción se completa lo que se entiende como **lanzar la excepción**. Cuando se crea el objeto del tipo *Exception* es habitual incluir en el información sobre el contexto en el que ha ocurrido, incluyendo una descripción textual sobre la causa que la ha provocado y/o referencias a objetos que se podrán usar en el tratamiento de la excepción. Al lanzar una excepción **se interrumpe** la ejecución del método, es decir, el flujo del programa abandona el método y apunta al código en el que se lleva a cabo el tratamiento de la excepción.

Las excepciones se lanzan con el comando *throw new Exception()*. La información del contexto de ocurrencia de la excepción se pasa en los argumentos del constructor de la clase asociada a la excepción

7.1.3. Tratar

Una vez se lanza una excepción, el objeto de tipo excepción se transfiere como “argumento” al código en el que se realiza su tratamiento. El momento en el que el flujo del programa entra en el código para el tratamiento de la excepción se denomina **captura de la excepción**.

El tratamiento de la excepción tienen lugar en los denominados bloques *catch{}*, que están asociados con *try{}* de modo que no puede haber un bloque *catch* sin un *try* previo. Un bloque *try* podrá tener tantos bloques *catch* como el número de tipos de excepciones que se puedan lanzar dentro del *try*. La opción **multi-catch** no añade ningún tipo de semántica a la gestión de la excepción, si no que está orientado a simplificar el código del bloque *catch*

```
catch( tipo_excepción | tipo_excepción | ... nombre_excepcion)
```

La herencia y el polimorfismo juegan un papel importante a la hora de decidir qué bloque *catch* se ejecutará cuando se lanza una excepción. Si ya se ha capturado una excepción de la clase base, no se podrá capturar una excepción de la clase derivada

En el tratamiento de las excepciones, existe el bloque *finally{}*, que se ejecutará siempre, independientemente de la excepción que se haya capturado. No es posible definir un bloque *finally{}* sin haber definido un bloque *catch{}*. El uso habitual del bloque *finally* es liberar los recursos

7.2. Revisión

Un método_A que invoca a un método_B que lanza una excepción no debe tener necesariamente un bloque *try-catch* como parte de su código, es decir, el método_A puede delegar el tratamiento de la excepción a otro método_C que lo invoque, de modo que existe un encadenamiento de excepciones¹

¹Ánimo, que este examen está chupadoo :))